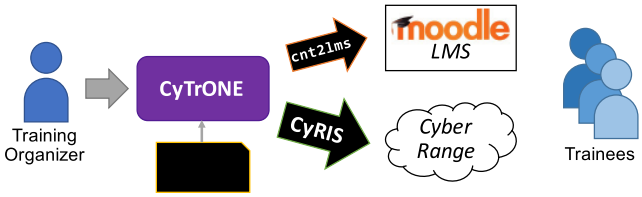

Fig. 1 Overview of the CyTrONE integrated cybersecurity training framework that is powered by CyRIS.

we are developing the CyTrONE training framework [2], for which CyRIS serves as a core component.

Figure 1 shows the overall architecture of CyTrONE. Based on input from the training organizer and a training database, CyTrONE uploads the training content to a Learning Management System (LMS) via the helper tool called `cnt2lms`, and also creates the associated training environment via CyRIS. Trainees can then access the LMS to consult the training content, connect to the cyber range to conduct the necessary investigation, and subsequently provide answers to questions via the LMS.

The training support system presented in this paper, CyRIS, is addressed to training organizers and instructors. However, CyRIS by itself does not provide a mechanism for interacting with the trainees. For this purpose, we have also developed the framework discussed above, which employs CyRIS for the creation and management of the training environment, but also relies on the LMS interface to address the needs of question display, trainee score management, etc. Basically, CyTrONE is a kind of meta-system, that is able to manage higher-level training operations, and delegates some lower-level tasks to other tools, in particular the cyber range instantiation task to CyRIS.

However, CyTrONE is only a use case for CyRIS, and any training program could use CyRIS as a back end, while having its own custom user interface (UI) for the participants. To use some examples from Japan, Hardening Project [3] is a security contest organized by the Web Application Security Forum, in which teams of IT professionals compete in terms of the service level they can provide for a virtual e-commerce company. Consequently, Hardening Project has no interface for learning, and uses a web UI for presenting contest results, such as the amount of virtual sales, or technical score. Another popular security contest in Japan, named SECCON [4], is a Capture The Flag (CTF) type of competition, and its custom UI presents users with challenges they have to solve in order to obtain points, shows their scores, etc. Although both these programs have different mechanisms for interaction with the participants, they could both use CyRIS as a back end to simplify the environment setup task.

2.1 Related Work

As for existing tools that are related to CyRIS, they can be divided into three categories, as discussed next:

1. *Configuration of individual nodes*: Installing and configuring programs, setting up the system, etc., but lacking features such as network service and topology configuration: Ansible [5], Vagrant [6], Chef [7];
2. *Configuration of node sets (cloud)*: Creating and managing either physical hosts or virtual machines (VMs) and the network service amongst them, however lacking features of configuring the settings and content of individual nodes: OpenStack [8], VMware vSphere [9] for VMs, SpringOS [10] for physical hosts;
3. *Full node and network configuration*: To the best of our knowledge, Alfons [11] is the only tool in this category, in particular related to security training, albeit not having security content generation functionality *per se*.

Given that Alfons is the system most similar to CyRIS, we shall briefly introduce it here. According to [11], Alfons was designed to address four main requirements: (i) facility management and virtual/physical node support; (ii) network parameter configuration; (iii) installation of content; (iv) environment separation on a network testbed. We conclude that Alfons features are mainly oriented towards environment setup; moreover, it is only able to install content that has been prepared in advance, but not to generate any new security content, e.g., through malware or cyberattacks.

As it will be described in the next section, CyRIS is a system that aims to combine the features of the above tools to provide the same usage convenience, but also provisions specific security features and training content generation, so as to offer additional flexibility to the organizers in the context of security training.

3. CyRIS Overview

The role of CyRIS is essential in the context of cybersecurity education and training, as it enables the automatic creation of cyber range training environments in a dynamic manner, and in a relatively short time, based on a cyber range description. Since it requires no significant effort and no advanced technical knowledge on the side of the organizers, the use of CyRIS makes it possible to conduct training sessions in various circumstances, and for a large number of trainees.

To facilitate the training activities in various organizations, CyRIS was released as open-source software via GitHub (`https://github.com/crond-jaist/cyris`). The only necessary resources for using CyRIS are one or more base VM images to be used for the cyber range guests—for which a sample is also available via GitHub—and an infrastructure for the cyber range, such as off-the-shelf servers.

Figure 2 presents an overview of the CyRIS architecture. The input to CyRIS are the cyber range description in YAML format, as well as VM disk images from the *VM Image Pool*, as specified in the cyber range description. The first processing stage of CyRIS is called *Base VM Preparation*, in which the base VM images to be employed are

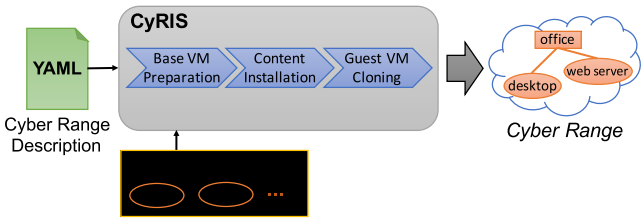

Fig. 2 Overview of the CyRIS input, output, and internal architecture.

copied from the pool and their basic setup is conducted. The second stage, called *Content Installation*, is the main processing step, when all the additional settings are applied, such as account creation, software installation, attack emulation, etc. The third and final stage is called *Guest VM Cloning*, and it refers to the actual deployment of multiple cyber range guest VMs based on the prepared base VM images.

To run CyRIS one has to deploy it on the servers to be used for cyber range creation, which should all use the Ubuntu OS. One of the servers plays the role of *master*, and manages the cyber range creation on all the other servers. Currently, CyRIS supports KVM virtualization technology, and the CentOS operating system for the cyber range guests; nevertheless, training environment setup features are also available for Ubuntu OS guests.

3.1 Cyber Range Description

The input format for CyRIS is based on the YAML format [12], with a series of custom tags used to define the characteristics of the cyber range to be created. Each file must contain the following sections:

- *Host settings*: Describe the host(s) on which the cyber range is to be created, such as the IP address used to access the host, the virtual bridge IP address, and the user account; a user-defined id is used to refer to the host elsewhere in the file;
- *Guest settings*: Describe the guest(s) the cyber range is composed of, with two classes of information:
  - *Base VM information*: Details about the base VM image that will be used to create the guest, such as the location of the base VM image file and its properties; a user-defined id is used to refer to the guest elsewhere in the file;
  - *Setup tasks*: Details about the setup operations that are to be performed on the guest during the cyber range instantiation process, such as user account operations, software installation, security incident emulation, etc.;
- *Clone settings*: Describe the manner in which the cyber range composed of the above guests is to be created, such as the total number of cyber range instances, the number of clones for each guest type, the interconnecting network topology, etc.

Table 1 Summary of the training environment setup and security content generation features of CyRIS, along with the related description tags.

Training Environment Setup

Function	*Description tag(s)*
Account Management	`add_account`; `modify_account`
Tool Installation	`install_package`
File Copy	`copy_content`
Program Execution	`execute_program`
Network Configuration	`topology`; `networks`; `members`

Security Content Generation

Function	*Description tag(s)*
Firewall Configuration	`firewall_rule`
Malware Emulation	`emulate_malware`
Attack Emulation	`emulate_attack`
Traffic Capture	`emulate_traffic_capture_file`

We summarize the most essential features of CyRIS in Table 1, together with the tags that are to be used in the cyber range description file in order to activate them. Since training environment setup functionality is easier to understand, we'll detail only the security content generation functions, which are specific to CyRIS. Thus, Firewall Configuration can be used to add predefined rules to the guest OS firewall, so that trainees can learn how to analyze such settings from a security point of view. Malware Emulation functionality can be used to run malware-like software; currently we include two dummy malware modules, one for creating an artificial CPU load, and one for listening to a given port, both providing the "symptoms" of malware without being actually malicious. Attack Emulation functionality includes three types of emulated attacks with configurable parameters, such as target or source IP, and so on: (i) SSH dictionary attack; (ii) DoS attack; (iii) DDoS attack; this functionality is further extensible via the program execution function, which allows running any real malware or attack tools. Finally, Traffic Capture makes it possible to store a network packet trace in a file for trainees to conduct forensics analysis on, for instance containing the traces of an attack conducted using the attack emulation features.

Figure 3 shows an example of a CyRIS cyber range description file. Its sections have the following content:

- In the section `host_settings`, the file defines a host with the id "host_1" and certain IP settings, that is to be accessed via the account "cyuser".
- Then, in the section `guest_settings`, it defines a guest with the id "dektop" to be created based on the configuration file "basevm_desktop.xml" located on "host_1". The tasks that are to be carried out to prepare the guest are: account creation for user "daniel"; package installation for "wireshark"; attack emulation via a number of 123 ssh login attempts; and malware emulation by means of a dummy process called "DAEMON" that will use 45% of CPU. An additional guest with the id "webserver" is also defined, which uses the base image configuration file "basevm_webserver.xml". Due to space limitations we only show one sample task, namely the execution of the

script "install_wordpress.sh" via the interpreter "bash";

- Finally, in the section `clone_settings`, the file describes that 30 instances of this cyber range will be created on "host_1". Each cyber range will include 1 "desktop" and 1 "webserver" guest, with the former being used as entry point for logging into the cyber range. Both guests will be connected to the network segment named "office" via the interface "eth0".

The example provided demonstrates that the cyber range description can be easily understood and modified if needed, either manually or through automated processing. The use of ids to refer to elements makes it easy to define the relationships between the logical components of the CyRIS cyber range model, such as hosts, guests, or networks. Moreover, users do not have to be concerned with low-level details, such as IP address settings, as these are carried out automatically by CyRIS depending on the composition and topology of the cyber range. We expect that, in the future, a GUI could be used to generate such description files in an even more convenient manner, thus eliminating the need to remember the keywords and file structure.

Usare gli LLM

3.2 Workflow

To facilitate the understanding of CyRIS, we present its detailed workflow in Fig. 4. The program starts by checking whether the input description file is syntactically and semantically correct. Then the *Base VM Preparation* stage is initiated, with steps such as copying the disk image(s) from the base VM pool, and starting them. If the start is successful, some basic setup operations are conducted, such as setting up ssh access, hostname, network connectivity, etc.

The second stage, *Content Installation*, consists first of all of performing all the setup tasks, such as installing content into the base VMs or emulating attacks; these steps are performed sequentially as described in the input file. The phase called "Post-cloning setup" however is carried out only *after* the cloning process ends; this is essential for performing configurations that depend on the properties of each cloned VM, such as its IP address, etc.

The third stage, *Guest VM Cloning*, refers to creating multiple cyber range instances that include guests based on the prepared based VMs—for use by trainees to conduct the same training simultaneously—and configuring their network topology. For this purpose, the configured base images are copied in parallel to all the hosts on which the cyber range instances are to be instantiated using the `parallel-scp` command; if a cyber range instance contains multiple base images, then they are also copied in parallel. After the cloned VMs are started, the user accounts and passwords for accessing the cyber range are randomly generated and the settings are applied. Note that network topology configuration between cloned VMs takes place at Layer 3, via IP address based routing through firewall rules. In the future, Layer 2 topology could also be configured via VLAN mechanisms.

```
---
- host_settings:
  - id: host_1
    mgmt_addr: 172.16.123.234
    virbr_addr: 192.168.122.1
    account: cyuser
- guest_settings:
  - id: desktop
    basevm_host: host_1
    basevm_config_file: /kvm/basevm_desktop.xml
    basevm_type: kvm
    tasks:
    - add_account:
      - account: daniel
        passwd: daniel_passwd
    - install_package:
      - package_manager: yum
        name: wireshark
    - emulate_attack:
      - attack_type: ssh_attack
        target_account: daniel
        attempt_number: 123
    - emulate_malware:
      - name: DAEMON
        cpu_utilization: 45
        mode: dummy_calculation
  - id: webserver
    basevm_host: host_1
    basevm_config_file: /kvm/basevm_webserver.xml
    basevm_type: kvm
    tasks:
    - execute_program:
      - program: /database/install_wordpress.sh
        interpreter: bash
- clone_settings:
  - range_id: 123
    hosts:
    - host_id: host_1
      instance_number: 30
      guests:
      - guest_id: desktop
        number: 1
        entry_point: yes
      - guest_id: webserver
        number: 1
      topology:
      - type: custom
        networks:
        - name: office
          members: desktop.eth0, webserver.eth0
```

Fig. 3 Example of a CyRIS cyber range description file.

During the entire setup process, CyRIS records the return values for all internal and external operations, and logs all the commands and their output. In case an error is detected from an invalid return value, an appropriate message is displayed. Otherwise, the cyber range creation is successful, and cyber range login information and internal details are provided to the user for reference.